

Corso: Linguaggi Formali e Compilatori

GeoScript

Documentazione Tecnica

Guida completa e approfondita alla programmazione di grafica vettoriale



Studenti:

Cesari Matteo [Mat. 1073570]

Girolamo Davide [Mat. 1073645]

Indice dei Contenuti

Panoramica	3
Scopo del Documento	3
Visione del Progetto	3
Ambito di Applicazione.....	3
Filosofia di Sviluppo e Architettura.....	4
Paradigma di Compilazione	4
Il Pattern Command	4
Disaccoppiamento Sintassi-Esecuzione	4
Vantaggi del Disaccoppiamento	5
Pipeline dei Dati	5
Stack Tecnologico	6
Core Technologies.....	6
Librerie e Dipendenze.....	6
Ambiente di Build.....	6
Modellazione UML e Design del Sistema	7
Diagramma di Distribuzione (Deploy Diagram).....	7
Diagramma dei Casi d'Uso (Use Case).....	9
Diagramma delle Classi (Class Diagram).....	10
Analisi Sintattica: La Grammatica	12
Convenzioni Lessicali.....	12
Grammatica Non Decorata (BNF)	12
Realizzazione dei Controlli Semantici	14
Controllo di Definizione dei Colori	14
Controllo di Esistenza delle Variabili	14
Controllo di Inizializzazione dell'Ambiente (Canvas)	15
Controlli Matematici (Divisione per Zero)	15
Controllo Validità Misure Geometriche	15

Panoramica

Scopo del Documento

Il presente documento ha l'obiettivo di fornire una disamina completa, dettagliata e strutturata dell'architettura interna del compilatore **GeoScript**.

È redatto specificamente per sviluppatori software e manutentori che necessitano di comprendere il funzionamento profondo del sistema per scopi di debugging, refactoring, manutenzione evolutiva o integrazione con sistemi terzi.

Visione del Progetto

GeoScript nasce per rendere più semplice e uniforme la creazione di grafica vettoriale generata tramite codice.

I formati attuali, come **SVG**, sono molto verbosi e statici; mentre le librerie grafiche complete richiedono interi ambienti di sviluppo.

GeoScript si propone come un DSL leggero, che permette di descrivere scene grafiche complesse usando costrutti semplici (variabili, cicli, condizioni) e di trasformarle automaticamente in un output vettoriale portabile.

Ambito di Applicazione

Il sistema è progettato per girare su qualsiasi macchina virtuale Java (JVM) e non richiede hardware grafico dedicato, poiché il rendering è puramente matematico e testuale (generazione di stringhe XML).

Filosofia di Sviluppo e Architettura

Paradigma di Compilazione

GeoScript non è un semplice traduttore "one-to-one". Adotta un approccio ibrido tra un compilatore e un interprete.

1. **Fase di Parsing:** Il codice sorgente viene validato sintatticamente.
2. **Fase di Intermediazione:** Non viene generato un AST (Abstract Syntax Tree) generico, ma una lista lineare di oggetti ad alto livello (Command List).
3. **Fase di Esecuzione:** La lista di comandi viene iterata ed eseguita da un motore semantico che mantiene lo stato del programma.

Il Pattern Command

La scelta architetturale cardine è l'utilizzo del Command Design Pattern.

Ogni istruzione del linguaggio (assegnazione variabile, disegno forma, ciclo) viene incapsulata in una classe Java che implementa l'interfaccia comune Command.

Questo offre vantaggi cruciali:

- **Incapsulamento:** Ogni comando contiene tutti i dati necessari per la sua esecuzione (es. ShapeCommand contiene le espressioni per le coordinate).
- **Differimento:** L'esecuzione non avviene durante il parsing. Questo permette di rilevare errori di sintassi nell'intero file prima di eseguire qualsiasi operazione (evitando file di output parziali o corrotti).
- **Riutilizzabilità:** I comandi possono essere salvati in liste (es. il corpo di un ciclo FOR) ed eseguiti molteplici volte senza dover ri-analizzare il testo sorgente.

Disaccoppiamento Sintassi-Esecuzione

L'architettura del sistema si basa su una netta separazione tra la fase di analisi sintattica (definita nel file GeoScript.g) e la fase di esecuzione semantica (gestita nel SemanticHandler.java).

- **Il Ruolo della Grammatica:** La grammatica funge esclusivamente da "interfaccia" del linguaggio. Il suo compito è convalidare la struttura del codice sorgente e mappare le istruzioni testuali in oggetti Java concreti
- **Il Ruolo del SemanticHandler (Logic Engine):** All'interno del SemanticHandler sono definite le classi che implementano l'interfaccia Command. Questi oggetti rappresentano la "logica di business" del compilatore: sanno come manipolare la memoria (variabili), come valutare le espressioni e come generare le stringhe SVG.

Vantaggi del Disaccoppiamento

Il collegamento tra le due componenti avviene solo tramite "azioni semantiche" (porzioni di codice Java inserite nella grammatica) che istanziano i comandi. Questo design garantisce un'elevata manutenibilità: se in futuro si volesse modificare la sintassi (ad esempio, passando da RECT AT a DRAW RECT), basterebbe aggiornare le regole grammaticali. La logica di esecuzione e la generazione dell'SVG rimarrebbero invariate, poiché gli oggetti Command sono agnostici rispetto alla forma testuale originale.

Pipeline dei Dati

Il flusso di trasformazione del dato è il seguente:

1. Source Code (.geo)
2. ANTLR Lexer
3. Token StreamANTLR Parser
4. List<Command>
5. Semantic Handler (Interpreter)
6. SVG Content Builder
7. Output File (.svg)

Stack Tecnologico

L'infrastruttura tecnologica è stata mantenuta intenzionalmente minimale per garantire portabilità e facilità di build.

Core Technologies

- **Java (JDK 1.8+):** Scelto per la robustezza, la tipizzazione statica e l'eccellente gestione delle stringhe (fondamentale per la generazione XML/SVG).
- **ANTLR (ANother Tool for Language Recognition) v3.5.3:** Il de-facto standard per la generazione di parser. La versione 3 è stata scelta per la sua semplicità nella generazione di codice Java diretto e leggibile all'interno della grammatica stessa, ideale per DSL di media complessità.

Librerie e Dipendenze

- **antlr-runtime-3.5.3.jar:** Unica dipendenza a runtime necessaria per far girare il Lexer e il Parser generati.
- **Nessuna libreria grafica esterna:** Il rendering SVG è gestito internamente tramite manipolazione di stringhe (StringBuilder), garantendo zero dipendenze da librerie come AWT o Swing.

Ambiente di Build

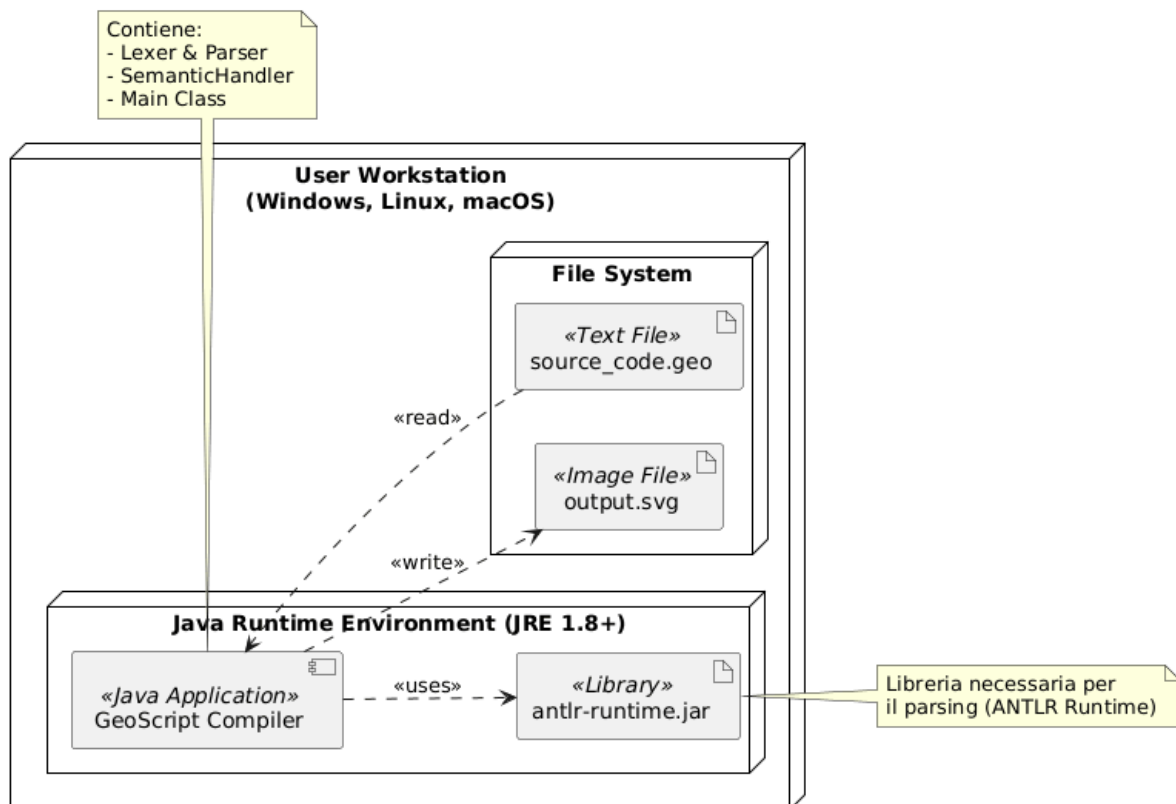
Il progetto non vincola a un IDE specifico. I file sorgente possono essere compilati tramite javac standard, previa generazione dei sorgenti ANTLR.

Modellazione UML e Design del Sistema

Di seguito sono riportati i diagrammi strutturali e comportamentali che descrivono il sistema.

Diagramma di Distribuzione (Deploy Diagram)

Il **Deploy Diagram** illustra la configurazione fisica dell'applicazione GeoScript durante l'esecuzione. Trattandosi di un compilatore stand-alone, l'architettura è centralizzata su un singolo nodo (la macchina dell'utente), senza dipendenze da server remoti o database distribuiti.



1. **Nodo Hardware:** User Workstation

L'ambiente di esecuzione è un personal computer standard. Non vi sono requisiti hardware specifici oltre a quelli minimi per eseguire il sistema operativo e la Java Virtual Machine.

- Sistemi Operativi Supportati: L'applicazione è multiplatforma (Windows, macOS, Linux) grazie all'astrazione fornita da Java.

2. **Ambiente di Esecuzione:** Java Runtime Environment (JRE)

Il cuore del sistema risiede all'interno della JVM (Java Virtual Machine).

- GeoScript Compiler (Component): È l'eseguibile principale (file .jar o classi compilate) che contiene il parser generato da ANTLR, il Lexer e il SemanticHandler.

- antlr-runtime.jar (Artifact): È l'unica dipendenza esterna critica. Questa libreria deve essere presente nel classpath affinché il compilatore possa interpretare l'albero sintattico generato dalla grammatica.

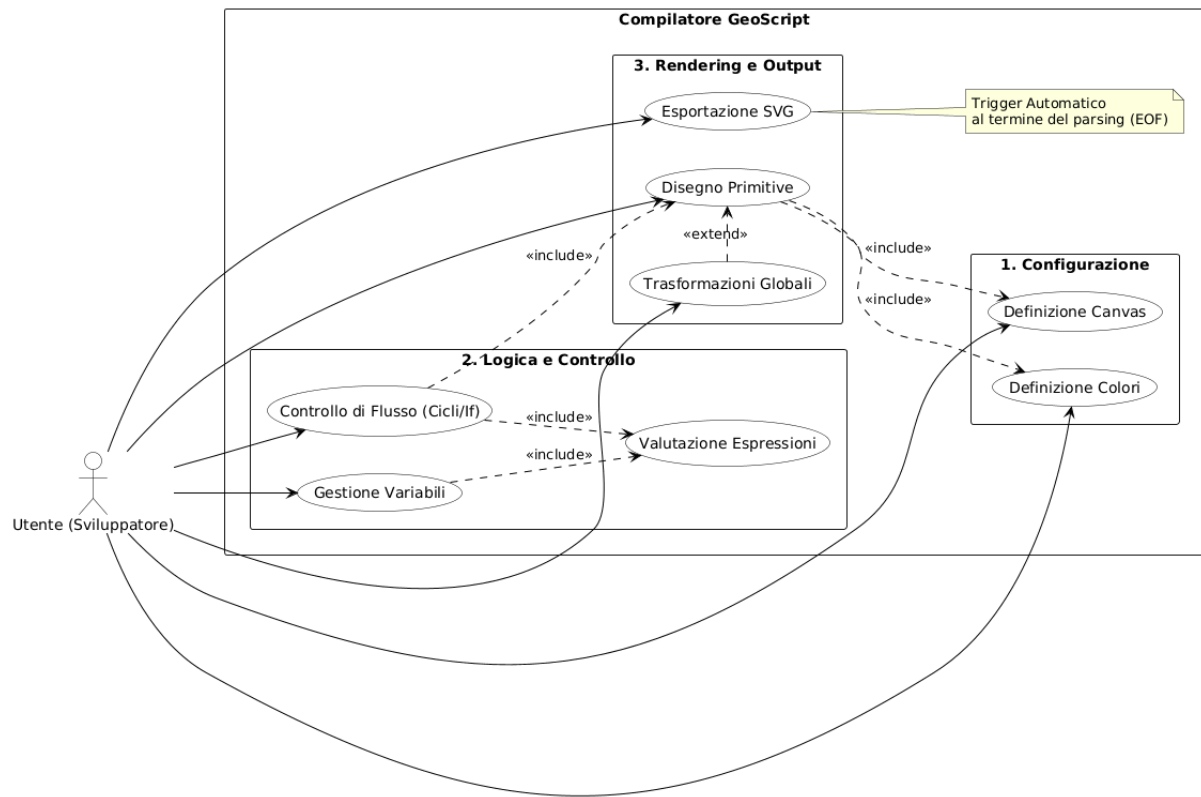
3. **Gestione degli Artefatti (File System)**

Il compilatore interagisce direttamente con il file system locale dell'utente per le operazioni di I/O:

- Input (.geo): Il file sorgente scritto dall'utente contenente le istruzioni GeoScript. Viene letto in modalità sola lettura.
- Output (.svg): Il risultato della compilazione. Viene generato (o sovrascritto) nella directory di esecuzione al termine del processo di parsing.

Diagramma dei Casi d'Uso (Use Case)

Questo diagramma descrive le interazioni tra l'utente (Attore) e il sistema GeoScript, evidenziando le funzionalità principali offerte dal linguaggio.

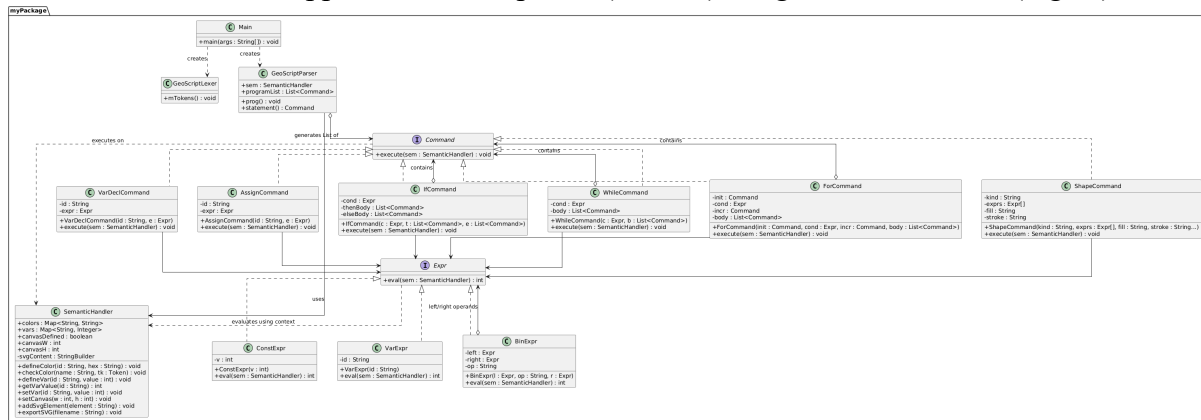


Descrizione dei casi d'uso

ID	Caso d'Uso	Descrizione Sintetica	Componente Responsabile
UC-01	Definizione Canvas	Impostazione delle dimensioni dell'area di lavoro (width, height).	SemanticHandler.setCanvas()
UC-02	Gestione Colori	Associazione di nomi mnemonici a codici esadecimali.	SemanticHandler.colors (Map)
UC-03	Gestione Variabili	Dichiarazione e assegnazione di valori interi.	VarDeclCommand, AssignCommand
UC-04	Controllo Flusso	Esecuzione condizionale (IF) o iterativa (FOR, WHILE).	IfCommand, WhileCommand, ForCommand
UC-05	Disegno Primitive	Creazione di forme geometriche (Rect, Circle, Polygon, etc.).	ShapeCommand
UC-06	Trasformazioni	Applicazione di rotazioni, scalature e traslazioni al contesto.	TransformCommand (Grammatica)
UC-07	Esportazione SVG	Scrittura finale del file grafico su disco.	SemanticHandler.exportSVG()

Diagramma delle Classi (Class Diagram)

Il diagramma delle classi illustra la struttura orientata agli oggetti del compilatore GeoScript, evidenziando il disaccoppiamento tra il parser (sintassi) e il gestore semantico (logica).



Struttura

1. GeoScriptParser

Situato nel package myCompilerPackage, il parser generato da ANTLR è responsabile della lettura del codice sorgente.

- Mantiene un riferimento al SemanticHandler (sem) per eseguire azioni immediate (come la definizione del canvas).
- Costruisce una lista di oggetti Command (programList) durante il parsing, popolandola man mano che riconosce statement validi.
- Al termine del parsing (prog), itera sulla lista ed esegue i comandi sequenzialmente.

2. SemanticHandler

Questa classe funge da Singleton di fatto per il contesto di esecuzione.

- Gestione Stato: Mantiene le tabelle dei simboli per le variabili (vars) e i colori (colors).
- Buffer Grafico: Accumula le stringhe SVG generate in svgContent e gestisce l'export finale su file tramite exportSVG.
- Classi Interne: Definisce le interfacce e le classi concrete per Comandi ed Espressioni, centralizzando la logica del linguaggio.

3. Pattern Command

L'interfaccia Command astrae l'esecuzione di qualsiasi istruzione.

- Le classi concrete (es. VarDeclCommand, ShapeCommand) incapsulano tutti i dati necessari per l'esecuzione.
- I comandi di controllo flusso (IfCommand, WhileCommand, ForCommand) implementano il pattern Composite, contenendo al loro interno liste di altri comandi (List<Command>), permettendo la nidificazione di blocchi di codice.

4. **Pattern Interpreter per Espressioni**

L'interfaccia Expr permette la valutazione a runtime di formule matematiche.

- BinExpr implementa una struttura ad albero ricorsiva per operatori binari (+, -, *, /, %), risolvendo espressioni complesse combinando ConstExpr (numeri) e VarExpr (variabili).

Analisi Sintattica: La Grammatica

Convenzioni Lessicali

Il Lexer è configurato per ignorare i whitespace (spazi, tabulazioni, a capo).

- **Commenti:** Supportati sia a linea singola (//) che multi-linea (/* ... */) e inviati al canale HIDDEN per non interferire con il parsing.
- **Identificatori (ID):** Alfanumerici, devono iniziare con lettera o underscore.
- **Colori:** Esadecimale rigoroso (# seguito da caratteri hex).

Grammatica Non Decorata (BNF)

Questa sezione presenta la struttura sintattica pura, utile per comprendere il linguaggio a colpo d'occhio senza il codice Java.

```
/* Struttura Principale */
<prog>          ::= { <statement> } EOF

<statement>     ::= <canvasStmt> | <colorDef> | <varDeclStmt> | <assignStmt>
                  | <ifStmt> | <whileStmt> | <forStmt> | <transformStmt>
                  | <shapeStmt>

/* Configurazione Ambiente */
<canvasStmt>    ::= "CANVAS" "(" INT "," INT ")" ";"
<colorDef>      ::= "DEF" ID "=" HEX_COLOR ";"

/* Gestione Variabili */
<varDeclStmt>   ::= "VAR" ID "=" <expr> ";"
<assignStmt>    ::= ID "=" <expr> ";"

/* Strutture Iterative e Condizionali */
<ifStmt>        ::= "IF" "(" <expr> ")" "THEN" "{" { <statement> } "}"
                  [ "ELSE" "{" { <statement> } "}" ]
<whileStmt>     ::= "WHILE" "(" <expr> ")" "{" { <statement> } "}"
<forStmt>       ::= "FOR" "(" [ <forInit> ] ";" <expr> ";" [ <forUpdate> ] ")" "{" {
<statement> } "}"
<forInit>       ::= "VAR" ID "=" <expr> | <assignSimple>
<forUpdate>     ::= <assignSimple>
<assignSimple>  ::= ID "=" <expr>
```

```

/* Trasformazioni Geometriche */
<transformStmt> ::= "ROTATE" INT ";"
                | "TRANSLATE" "(" <expr> "," <expr> ")" ";"
                | "SCALE" "(" <expr> "," <expr> ")" ";"

/* Primitive Grafiche e Testo */
<shapeStmt>      ::= "RECT" "AT" "(" <expr> "," <expr> ")" "SIZE" "(" <expr> "," <expr> ")"
<style> ";"
                | "CIRCLE" "AT" "(" <expr> "," <expr> ")" "RADIUS" <expr> <style> ";"
                | "LINE" "FROM" "(" <expr> "," <expr> ")" "TO" "(" <expr> "," <expr> ")"
"STROKE" <color> ";"
                | "SQUARE" "AT" "(" <expr> "," <expr> ")" "SIZE" <expr> <style> ";"
                | "TRIANGLE" "AT" "(" <expr> "," <expr> ")" "POINTS" "(" <expr> ","
<expr> "," <expr> "," <expr> "," <expr> "," <expr> ")" <style> ";"
                | "ELLIPSE" "AT" "(" <expr> "," <expr> ")" "RADII" "(" <expr> "," <expr>
")" <style> ";"
                | "POLYGON" "POINTS" "(" <pointList> ")" <style> ";"
                | "TEXT" "(" STRING "," <expr> "," <expr> ")" [ "COLOR" <color> ] ";"

<style>          ::= [ "FILL" <color> ] [ "STROKE" <color> ]
<color>          ::= ID | HEX_COLOR
<pointList>      ::= <expr> "," <expr> { "," <expr> "," <expr> }

/* Gerarchia delle Espressioni */
<expr>           ::= <addExpr> { ( "<" | ">" | "==" | "!=" ) <addExpr> }
<addExpr>        ::= <term> { ( "+" | "-" ) <term> }
<term>           ::= <factor> { ( "*" | "/" | "%" ) <factor> }
<factor>         ::= INT | ID | "(" <expr> ")"

```

Realizzazione dei Controlli Semantici

I controlli semantici nel compilatore GeoScript sono implementati principalmente all'interno della classe `SemanticHandler.java` e vengono eseguiti a runtime (durante la fase di esecuzione dei comandi), sfruttando le informazioni memorizzate nelle tabelle dei simboli (Mappe).

Di seguito sono descritti i tre meccanismi di controllo principali:

Controllo di Definizione dei Colori

Il sistema verifica che ogni colore utilizzato nelle forme (es. `FILL rosso`) sia stato precedentemente definito con un comando `DEF`.

- **Metodo:** `checkColor(String name, Token tk)`
- **Logica:**
 - Il metodo riceve il nome del colore.
 - Verifica se il nome inizia con `#` (esadecimale diretto). Se sì, il controllo passa.
 - Se è un nome mnemonico, consulta la mappa `colors`.
- **Azione di Errore:** Se la chiave non esiste, stampa un messaggio di warning su `System.err` indicando la riga dell'errore (recuperata dal `Token`).
- **Implementazione:**

```
public void checkColor(String name, Token tk) {  
    if (!colors.containsKey(name))  
        System.err.println("Warning: color '" + name + "' not defined at  
line " + tk.getLine());  
}
```

Controllo di Esistenza delle Variabili

Il sistema impedisce l'uso o l'assegnazione di variabili non dichiarate per evitare comportamenti imprevisti. Questo controllo avviene in due momenti:

- **In Lettura** (`getVarValue`):
Quando un'espressione (`VarExpr`) richiede il valore di una variabile, il sistema controlla la mappa `vars`.
Gestione Errore: Se la variabile non esiste, viene segnalato un warning e restituito un valore di default (0) per evitare il crash dell'applicazione (approccio fault-tolerant).
- **In Scrittura** (`setVar`):
Il comando `AssignCommand` (`x = 5;`) chiama `setVar`.
Gestione Errore: Se si tenta di assegnare valore a una variabile mai dichiarata con `VAR`, viene stampato un warning.

Controllo di Inizializzazione dell'Ambiente (Canvas)

Il sistema impedisce la generazione di codice SVG se l'area di lavoro non è stata definita, poiché le coordinate non avrebbero un riferimento valido.

- **Flag di Stato:** Viene utilizzato un booleano `canvasDefined` nel `SemanticHandler`.
- **Esecuzione:** All'inizio del metodo `execute()` di ogni `ShapeCommand` (es. rettangoli, cerchi), viene verificato questo flag.
- **Eccezione:** Il comando `TEXT` è esente da questo blocco rigoroso in alcune implementazioni, ma per le forme geometriche il controllo è bloccante:

```
if (!sem.canvasDefined && !kind.equals("TEXT")) return;
```

Se il canvas non è definito, il comando viene ignorato silenziosamente (o potrebbe generare un errore, a seconda della configurazione desiderata), evitando la scrittura di tag SVG invalidi.

Controlli Matematici (Divisione per Zero)

All'interno della classe `BinExpr`, durante la valutazione delle espressioni, viene effettuato un controllo semantico sulle operazioni di divisione (/) e modulo (%).

- **Logica:** Se il divisore (`right.eval(s)`) è 0, l'operazione non viene eseguita e viene restituito 0 (o gestito diversamente) per evitare l'eccezione `ArithmeticException` di Java che arresterebbe il compilatore.

Controllo Validità Misure Geometriche

Il sistema impedisce la creazione di figure con dimensioni negative (es. raggio o lati negativi), che non avrebbero senso geometrico o visuale e genererebbero SVG non validi.

- **Metodo:** Eseguito direttamente all'interno del metodo `execute` della classe `ShapeCommand`, subito dopo la valutazione delle espressioni.
- **Logica:** Il sistema analizza il tipo di forma richiesta (`RECT`, `CIRCLE`, `SQUARE`, `ELLIPSE`). Verifica che i valori dimensionali calcolati siano non negativi: larghezza e altezza per i rettangoli, raggio per i cerchi, lato per i quadrati, e raggi x/y per le ellissi.
- **Azione di Errore:** Se viene rilevato un valore negativo, viene stampato un messaggio di errore specifico su `System.err` (es. "Error: RECT dimensions cannot be negative") e l'esecuzione del singolo comando viene interrotta (`return`). Questo evita che venga generato codice SVG errato per quella specifica forma, permettendo comunque al resto del programma di proseguire.